

CodeReview-Parsa-Ian

Ian

Smells

- Duplicate Code - **randomValue** in **Item Spawn Methods**

Spawn methods localized to SpawnHelper, this includes both the value assignment and the location assignment of new Items

- Duplicate Code - **collectIfTouched** in **Item Classes**

Collection handling and methods are localized to the master Item class.

- Long Class - Separate out **paintComponent**

Removed paintComponent class components from GamePanel class and into it's own Game_Painting class.

- Static Variables - **Item Counts** and **Item Limits**

Item counts are now in the update file with a simple if statement. Limits and whatnot will be localized to master Item Class

Parsa

Smell: Duplicated code in score-saving logic

Problem: The logic for saving the final score appeared both inside update() and inside saveScoreOnce(). This duplication made the code harder to maintain and increased the risk of inconsistent behavior if one copy was changed without the other.

Refactoring: We removed the duplicate score-saving block from update() and reused saveScoreOnce() as the single method responsible for saving the score exactly once.

Smell: Data Clump

Problem: The variables playerWidth and playerHeight were always used together and had identical values. This indicates a data clump, making the code redundant and harder to maintain.

Refactoring: We replaced both variables with a single constant PLAYER_SIZE, simplifying method calls and reducing duplication while preserving behavior.

1. Long Constructor

One of the code smells in GamePanel was the constructor being too long. It was doing many different things at once, such as creating the map and spawners, loading images, creating the painter, resetting the game state, setting up the panel, adding keyboard

input, and starting the timer. Because all of this logic was placed in one constructor, it was harder to read and understand what the main purpose of each part was.

To improve this, I used extract method refactoring and moved the setup code into smaller helper methods: `initializeDependencies()`, `loadAssets()`, `initializePainterAndState()`, `configurePanel()`, `setupInputHandling()`, and `startGameLoop()`. This made the constructor much cleaner and easier to follow, since it now shows the overall setup process more clearly without changing how the game works.

Commit: Clean up GamePanel constructor by moving setup code into helper methods

2. Long / Complex Method

Another smell was the keyboard handling logic inside `keyPressed(...)`. It had several if statements handling different cases, including showing highscores, starting the game, restarting, pausing, and allowing movement. Since all of these checks were written in one block, the method was harder to read and more difficult to update safely.

I refactored this by splitting the logic into smaller methods:

`handleKeyPressed(int key)`, `handleMenuKey(int key)`, `handleControlKey(int key)`, and `isGameplayInputAllowed()`. This made the flow of the code easier to understand because each method now has one clear purpose. The behavior stayed the same, but the structure became more organized.

Commit: Break up GamePanel keyboard handling into smaller methods

3. Primitive Obsession / Magic Numbers

The last smell I focused on was the use of a magic number in the game loop. In `startGameLoop()`, the timer delay was written directly as 16. Even though the value is small, it is better to avoid hardcoded numbers like this because the meaning is not immediately obvious when someone reads the code.

To fix this, I replaced the hardcoded 16 with a named constant called `GAME_LOOP_DELAY_MS`. This makes the code easier to understand because the purpose of the number is now clear. It also makes future changes simpler, since the value only needs to be updated in one place.

Commit: Replace GamePanel timer magic number with a named constant

Testing

After making these changes, I ran the project checks to make sure the refactoring did not break GamePanel. The code compiled successfully with mvn clean compile, and the targeted GamePanel tests passed with:

```
mvn test "-Dtest=GamePanelStateTest,GamePanel_Test"
```

This confirmed that the changes improved the code structure without changing the intended behavior.